(1) The circuit diagram of your design and explaining your design (You can use screenshot to explain)

DCIM

```
*****************
*** DCIM block ***
*****************
.subckt DCIM I1 I2 I3 I4 IN_VAL CLK RST O10 O11 O12 O13 O14 O15 O16 O17 O18 O19 O20 O21 O22 O23 O24 O25 O26 O27 O28 O29 OUT_VAL
Xcontrol_in I1 I2 I3 I4 CLK IN_VAL OUT_VAL ii1 ii2 ii3 ii4 CON_IN
Xcontrol_out1 P10 P11 P12 P13 P14 P15 P16 P17 P18 P19 CLK RST O10 O11 O12 O13 O14 O15 O16 O17 O18 O19 CON_OUT
Xcontrol_out2 P20 P21 P22 P23 P24 P25 P26 P27 P28 P29 CLK RST O20 O21 O22 O23 O24 O25 O26 O27 O28 O29 CON_OUT
Xmain1 ii1 ii2 ii3 ii4 CLK O10 O11 O12 O13 O14 O15 O16 O17 O18 O19 P10 P11 P12 P13 P14 P15 P16 P17 P18 P19
+ w0_0 w1_0 w2_0 w3_0 w4_0 w5_0 w6_0 w7_0 w8_0 w9_0 w10_0 w11_0 w12_0 w13_0 w14_0 w15_0 Main
Xmain2 ii1 ii2 ii3 ii4 CLK O20 O21 O22 O23 O24 O25 O26 O27 O28 O29 P20 P21 P22 P23 P24 P25 P26 P27 P28 P29
+ w0_1 w1_1 w2_1 w3_1 w4_1 w5_1 w6_1 w7_1 w8_1 w9_1 w10_1 w11_1 w12_1 w13_1 w14_1 w15_1 Main
.ends
```

這裡會處理所有計算邏輯

CON_IN

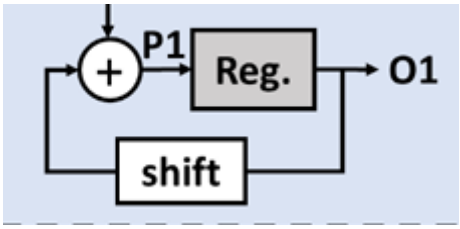
```
.subckt CON_IN I1 I2 I3 I4 CLK IN_VAL OUT_VAL ii1 ii2 ii3 ii4
Xff1 I1 CLK vdd ii1 FF
Xff2 I2 CLK vdd ii2 FF
Xff3 I3 CLK vdd ii3 FF
Xff4 I4 CLK vdd ii4 FF
Xinv IN_VAL IN_VAL_inv INV
Xff5 IN_VAL_inv CLK vdd OUT_VAL FF
.ends
```

這裡是在把四個 Input 傳入 Flip Flop 裡面，下一個 Positive edge 會寫入 ii1、ii2、ii3、ii4

CON_OUT

```
.subckt CON_OUT P10 P11 P12 P13 P14 P15 P16 P17 P18 P19 CLK RST O10 O11 O12 O13 O14 O15 O16 O17 O18 O19
* Flip-flops for each bit with positive edge triggering and active-low reset
Xff10 P10 CLK RST O10 FF
Xff11 P11 CLK RST O11 FF
Xff12 P12 CLK RST O12 FF
Xff13 P13 CLK RST O13 FF
Xff14 P14 CLK RST O14 FF
Xff15 P15 CLK RST O15 FF
Xff16 P16 CLK RST O16 FF
Xff17 P17 CLK RST O17 FF
Xff18 P18 CLK RST O18 FF
Xff19 P19 CLK RST O19 FF
.ends
```

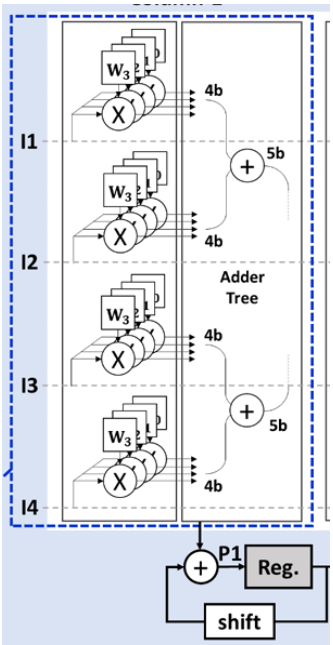
這裡會把電路圖下方的 P 傳入 Flip Flop 裡面，下一個 Positive edge 會寫入 10-bit 的 Output，如下圖



Main

```
*** Main Circuit
.subckt Main I1 I2 I3 I4 CLK O0 O1 O2 O3 O4 O5 O6 O7 O8 O9 P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 w0 w1 w2 w3 w4 w5 w6 w7 w8 w9 w10 w11 w12 w13 w14 w15
* Step 1: Multiply each input (I1, I2, I3, I4) by the 4-bit weights (W)
Xlatch_mul_4bit1 w0 w1 w2 w3 I1 P1_0 P1_1 P1_2 P1_3 LATCH_MULT_4BIT
Xlatch_mul_4bit2 w4 w5 w6 w7 I2 P2_0 P2_1 P2_2 P2_3 LATCH_MULT_4BIT
Xlatch_mul_4bit3 w8 w9 w10 w11 I3 P3_0 P3_1 P3_2 P3_3 LATCH_MULT_4BIT
Xlatch_mul_4bit4 w12 w13 w14 w15 I4 P4_0 P4_1 P4_2 P4_3 LATCH_MULT_4BIT
* Step 2: Add I1's and I2's outputs (4-bit each) to get 5-bit
Xadd1 P1_0 P1_1 P1_2 P1_3 P2_0 P2_1 P2_2 P2_3 vss S12_0 S12_1 S12_2 S12_3 S12_4 ADDER_4BIT
* Add I3's and I4's outputs (4-bit each) to get another 5-bit
Xadd2 P3_0 P3_1 P3_2 P3_3 P4_0 P4_1 P4_2 P4_3 vss S34_0 S34_1 S34_2 S34_3 S34_4 ADDER_4BIT
* Step 3: Add the two 5-bit results to get 6-bit output (output6)
Xadd3 S12_0 S12_1 S12_2 S12_3 S12_4 S34_0 S34_1 S34_2 S34_3 S34_4 vss O6_0 O6_1 O6_2 O6_3 O6_4 O6_5 ADDER_5BIT_6BIT
* Step 4: Left shift the 10-bit value stored in Flip-Flop
* Step 5: Add the shifted 10-bit value and output6 (6-bit)
Xadd10 vss O0 O1 O2 O3 O4 O5 O6 O7 O8 O6_5 O6_4 O6_3 O6_2 O6_1 O6_0 vss vss vss vss vss P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 ADDER_10BIT
.ends
```

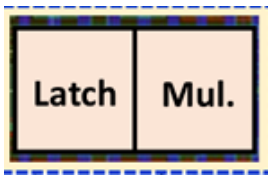
這裡在處理乘法、所有加法、Bit Shifting 等



LATCH_MULT_4BIT

```
.subckt LATCH_MULT_4BIT W0 W1 W2 W3 IN OUT0 OUT1 OUT2 OUT3
Xinv In In_inv INV
Xlatch0 In_inv W0 OUT0 LATCH_MULT
Xlatch1 In_inv W1 OUT1 LATCH_MULT
Xlatch2 In_inv W2 OUT2 LATCH_MULT
Xlatch3 In_inv W3 OUT3 LATCH_MULT
.ends
```

為了方便，我寫了一個 Latch +乘法的 subckt，一次處理 4 bits，有四個下面這個東西



各種 ADDER (4-bit、5-bit、10-bit)

```
*** 4-bit + 4-bit = 5-bit ***
.subckt ADDER_4BIT A0 A1 A2 A3 B0 B1 B2 B3 Cin S0 S1 S2 S3 S4
Xfa0 A3 B3 Cin S4 c1 FULL_ADDER
Xfa1 A2 B2 c1 S3 c2 FULL_ADDER
Xfa2 A1 B1 c2 S2 c3 FULL_ADDER
Xfa3 A0 B0 c3 S1 S0 FULL_ADDER
.ends

*** 5-bit + 5-bit = 6-bit ***
.subckt ADDER_5BIT_6BIT A0 A1 A2 A3 A4 B0 B1 B2 B3 B4 Cin S0 S1 S2 S3 S4 S5
Xfa0 A4 B4 Cin S5 c1 FULL_ADDER
Xfa1 A3 B3 c1 S4 c2 FULL_ADDER
Xfa2 A2 B2 c2 S3 c3 FULL_ADDER
Xfa3 A1 B1 c3 S2 c4 FULL_ADDER
Xfa4 A0 B0 c4 S1 S0 FULL_ADDER
.ends

*** 10-bit adder ***
.subckt ADDER_10BIT A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 Cin S0 S1 S2 S3 S4 S5 S6 S7 S8 S9
Xfa0 A0 B0 Cin S0 c1 FULL_ADDER
Xfa1 A1 B1 c1 S1 c2 FULL_ADDER
Xfa2 A2 B2 c2 S2 c3 FULL_ADDER
Xfa3 A3 B3 c3 S3 c4 FULL_ADDER
Xfa4 A4 B4 c4 S4 c5 FULL_ADDER
Xfa5 A5 B5 c5 S5 c6 FULL_ADDER
Xfa6 A6 B6 c6 S6 c7 FULL_ADDER
Xfa7 A7 B7 c7 S7 c8 FULL_ADDER
Xfa8 A8 B8 c8 S8 S9 FULL_ADDER
.ends
```

剩下的部分是一些更小的 **Component**，負責兜出上面所有的邏輯

LATCH、LATCH_MULT

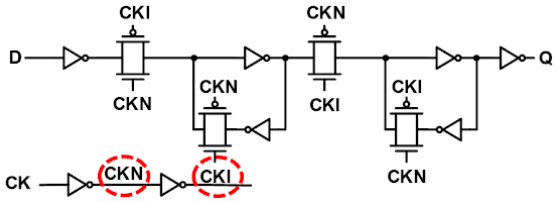
```
.subckt LATCH IN OUT
Xinv1 IN OUT INV
Xinv2 OUT IN INV
.ends

.subckt LATCH_MULT IN W OUT
* LATCH *
XLatch W W_inv LATCH
* MUL *
Xnor1 W_inv IN OUT NOR2
.ends
```

LATCH_MULT: 將 Latch 的 Output 與輸入值 IN 做乘法，這邊直接用 nor 節省 transistor

Transmission Gate(TG)、LATCH with TG、Flip Flop

```
*** Transmission Gate ***
.subckt TG S S_inv input OUT
mp1 input S_inv OUT vdd P_18 w=0.72u l=0.18u
mn1 OUT S input vss N_18 w=0.36u l=0.18u
.ends

*** LATCH with TG ***
.subckt LATCH_TG IN CLK CLK_inv OUT
Xinv1 IN OUT INV
Xinv2 OUT OUT_inv INV
XTG CLK CLK_inv OUT_inv IN TG
.ends

.subckt FF D CLK RST Q
Xand1 D RST D_rst AND2
Xinv CLK CLK_inv INV
XTG1 CLK_inv CLK D_rst o1 TG
XLATCH_TG1 o1 CLK CLK_inv o1_inv LATCH_TG
XTG2 CLK CLK_inv o1_inv o2 TG
XLATCH_TG2 o2 CLK_inv CLK Q LATCH_TG
```

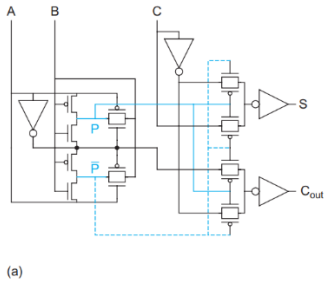
其中 Flip Flop 的架構採用下圖



FULL_ADDER

```
*** Full Adder ***
.subckt FULL_ADDER A B C S Cout
Xinv1 A A_inv INV
Xinv2 C C_inv INV
mp1 A B P vdd P_18 w=0.72u l=0.18u
mn1 P B A_inv vss N_18 w=0.36u l=0.18u
mp2 A_inv B P_inv vdd P_18 w=0.72u l=0.18u
mn2 P_inv B A vss N_18 w=0.36u l=0.18u
Xtg1 A_inv A B P TG
Xtg2 A A_inv B P_inv TG

Xtg3 P_inv P C_inv S_inv TG
Xtg4 P P_inv C S_inv TG

Xtg5 P_inv P A_inv Cout_inv TG
Xtg6 P P_inv C_inv Cout_inv TG

Xinv3 S_inv S INV
Xinv4 Cout_inv Cout INV
.ends
```

採用的架構如下圖，相較於傳統 Full Adder，能使用更少的 Transistor



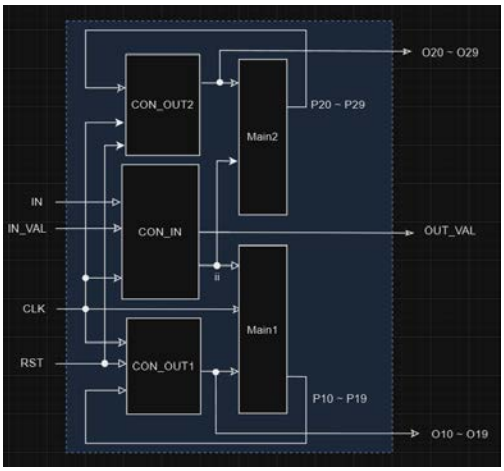
(a)

INV、AND2、NOR2、NAND2

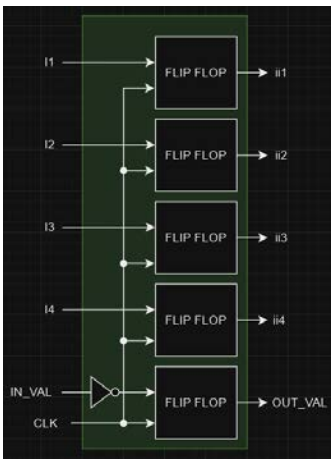
```
*** Inverter ***
.subckt INV in1 inv_out
mp1 vdd in1 inv_out vdd P_18 w=0.72u l=0.18u
mn1 inv_out in1 vss vss N_18 w=0.36u l=0.18u
.ends

*** AND ***
.subckt AND2 in4 in5 AND
Xnand in4 in5 nand_output NAND2
Xinv nand_output AND INV
.ends

*** NOR ***
.subckt NOR2 in1 in2 n1
mp1 vdd in1 n vdd P_18 w=0.72u l=0.18u
mp2 n in2 n1 vdd P_18 w=0.72u l=0.18u
mn1 n1 in1 vss vss N_18 w=0.36u l=0.18u
mn2 n1 in2 vss vss N_18 w=0.36u l=0.18u
.ends

*** NAND ***
.subckt NAND2 in1 in2 n1
mp1 vdd in1 n1 vdd P_18 w=0.72u l=0.18u
mp2 vdd in2 n1 vdd P_18 w=0.72u l=0.18u
mn1 n in1 vss vss N_18 w=0.36u l=0.18u
mn2 n1 in2 n vss N_18 w=0.36u l=0.18u
.ends
```

(2) The transistor level view of your CIM circuit and adder (You can just draw one of them and specified how you connect each)

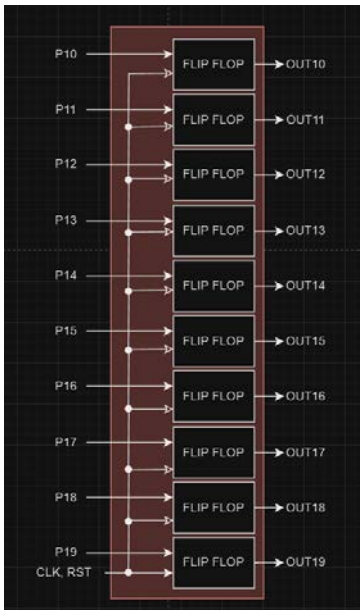
DCIM



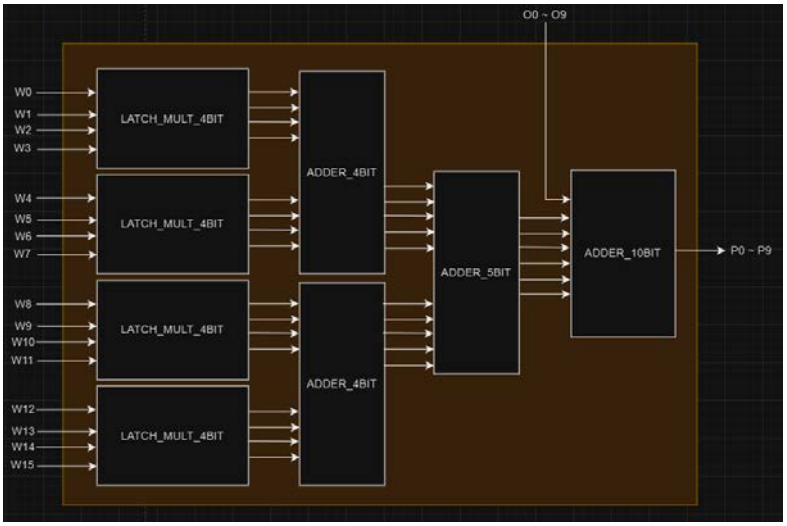
CON_INV



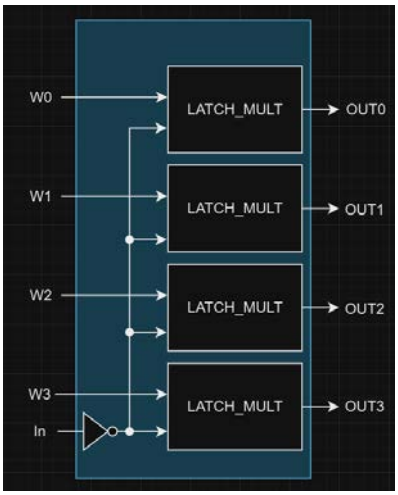
CON_OUT



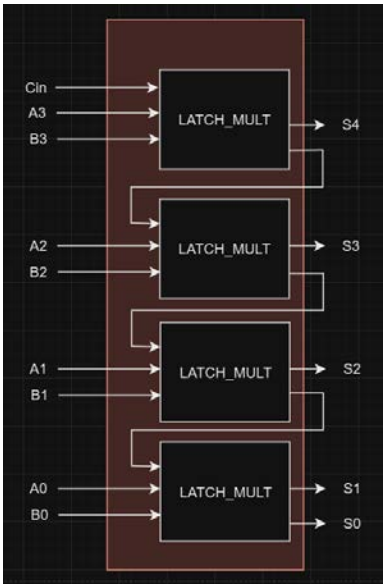
Main



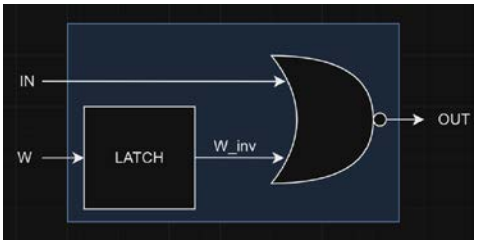
LATCH_MULT_4BIT



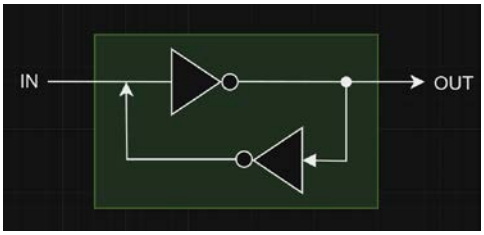
各種 ADDER (4-bit) (5-bit、10-bit 邏輯相同故省略)



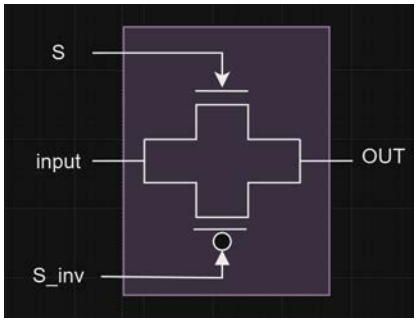
LATCH_MULT



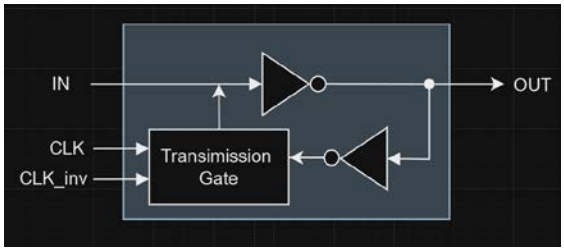
LATCH



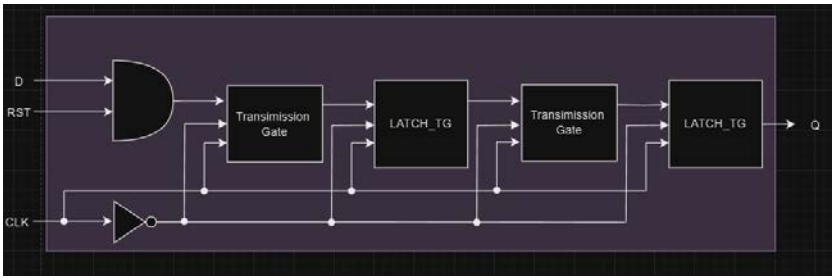
Transmission Gate



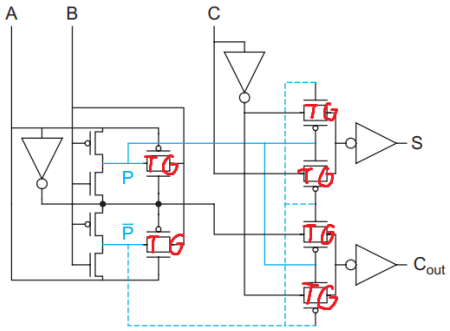
LATCH with TG



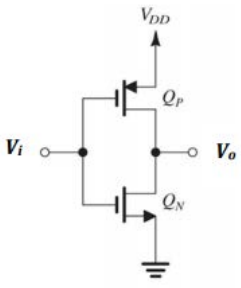
Flip Flop



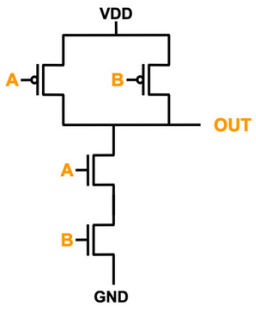
FULL_ADDER

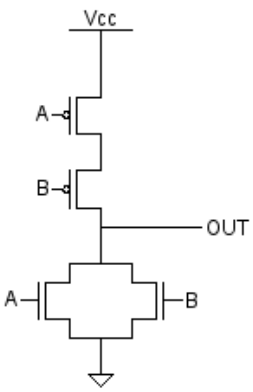


INV

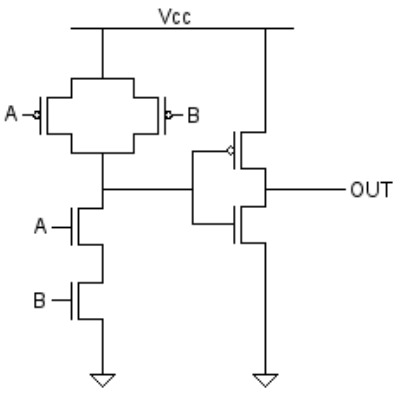


NAND





NOR



AND

(3) The bonus circuit if implemented

No

(4) Waveform of your simulation

O10~O19

O20~O29

(5) The delay and the power of the circuit

776.3825p、1.4664m

```
******
** dcim **

****** transient analysis tnom=  25.000 temp=  30.000 ******
td= 776.3825p  targ=  4.8728n  trig=  4.0964n
pwr=  1.4664m  from=  0.  to=  26.0000n

        ***** job concluded
******
** dcim **
```

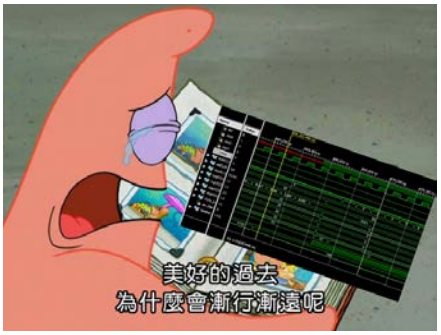
(6) The total number of transistors (NMOS and PMOS) you use in this program

1930

```
******  Circuit Statistics  ******
# nodes       =    4609 # elements    =    1939
# resistors   =       0 # capacitors =       0 # inductors   =       0
# mutual_inds =       0 # vccs       =       0 # vcvs        =       0
# cccs        =       0 # ccvs       =       0 # volt_srcs   =       9
# curr_srcs   =       0 # diodes     =       0 # bjts        =       0
# jfets       =       0 # mosfets    =    1930 # U elements  =       0
# T elements  =       0 # W elements =       0 # B elements  =       0
# S elements  =       0 # P elements =       0 # va device   =       0
# vector_srcs =       0 # N elements =       0
```

(7) The hardness of this assignment and how you overcome it

最難的是怎麼 Debug，大概有 90% 的時間都在看波形圖懷疑人生。我解決的方法就是每次都從頭比對到尾，深怕有任何 bug 害他壞掉，寫到後面都快把波型圖背起來了。



(8) Any suggestions about this programming assignment

不確定助教會怎麼測試檔案